

Министерство образования Республики Беларусь  
Учреждение образования  
«Брестский государственный технический университет»  
Кафедра ИИТ

Лабораторная работа №1  
По дисциплине: «КиСБ»  
Тема: «Генерирование равномерно распределенных псевдослучайных  
последовательностей»

Выполнил:  
Студент 4-го курса  
Группы АС-66  
Горобец А.В.  
Проверил:  
Хацкевич А.С.

Брест 2026

### Вариант 3

Цель работы: освоить основные алгоритмы программного генерирования равномерно распределенных псевдослучайных последовательностей.

Задание:

#### Часть 1:

1. На основе информационного сообщения, представленного символами русского/английского алфавитов, служебными символами и цифрами, содержащегося в некотором текстовом файле (согласовать с преподавателем), сформировать информационное сообщение в двоичном виде; длина сообщения в бинарном виде должна быть не менее 16 символов. Для выполнения этого задания можно использовать коды ASCII символов алфавита либо результаты лабораторной работы №2. 2. Для полученного информационного слова построить проверочную матрицу Хемминга (значение минимального кодового расстояния согласовать с преподавателем). 3. Используя построенную матрицу, вычислить избыточные символы (слово X). 4. Принять слово следующим числом ошибок: 0, 1, 2. Поскольку ошибки определяется (генерируется) случайным образом. 5. Для полученного слова  $U_n = U_k, Y_r$ , используя уже известную проверочную матрицу Хемминга, вновь вычислить избыточные символы (обозначим их  $\Gamma$ ), используя выражение (4.6). 6. Вычислить и проанализировать синдром. В случае, если анализ синдрома показал, что было передано с ошибкой (или 2 ошибками), унарный вектор ошибки  $E_n = \psi, \phi$ , и исправить одиночную ошибку, используя формулу (4.7); проанализировать ситуацию при возникновении ошибки в 2 битах. 7. Результаты оформить в виде отчета по установленным правилам. Файл анализируемой информации (а также ответственно и интерфейс приложения) должен содержать исходное информационное сообщение, значения величин  $K, r, n$ , проверочную матрицу Хемминга слова  $X, X_n, Y_n, Y_r, Y_r'$ , синдром  $S$ , вектор ошибки  $E_n$ . Программа не должна быть чувствительна к длине информационного сообщения. Некоторой сложностью, с которой можно столкнуться при выполнении данной лабораторной работы, является выбор и реализация алгоритма построения проверочной матрицы Хемминга. Существует несколько способов, позволяющих заполнить матрицу Хемминга. Одним из простых является следующий. Поскольку известно, что минимальный вес столбцов в подматрице  $P'$  (см. выражение (4.5)) равен 2, вычисляем значение бинарного Ньютона по следующей формуле:

$$C_{wt}^r = \frac{r!}{wt(r-wt)!},$$

где  $wt = 2$ , а  $r = \log_2 k + 1$ ;  $wt$  – это вес (количество единиц).

```

import java.util.*;
public class Lab2p1 {

    static class HammingCode {
        int k;      // длина информационной части
        int r;      // количество проверочных бит
        int n;      // общая длина кодового слова ( $n = k + r$ )
        int[][] H;  // проверочная матрица  $r \times n$ 
        int[][] G;  // порождающая матрица  $k \times n$  (для справки)

        /**
         * Конструктор: подбирает минимальное  $r$  для заданного  $k$ 
         * и строит систематическую проверочную матрицу.
         */
        HammingCode(int infoLength) {
            this.k = infoLength;
            // Ищем минимальное  $r$ :  $2^r \geq k + r + 1$ 
            r = 1;
            while ((1 << r) < k + r + 1) {
                r++;
            }
            n = k + r;

            buildParityCheckMatrix();
            // buildGeneratorMatrix(); // можно раскомментировать при необходимости
        }

        /**
         * Строим проверочную матрицу  $H$  в систематической форме:
         *  $H = [P \mid I_r]$ 
         * Столбцы — все различные ненулевые двоичные векторы длины  $r$ ,
         * последние  $r$  столбцов образуют единичную матрицу.
         */
        private void buildParityCheckMatrix() {
            H = new int[r][n];

            // Сначала заполняем единичную матрицу в конце (позиции  $k \dots n-1$ )
            for (int i = 0; i < r; i++) {
                H[i][k + i] = 1;
            }

            // Теперь заполняем первые  $k$  столбцов различными ненулевыми векторами,
            // которые ещё не использованы (не являются базисными векторами единичной матрицы)
            List<int[]> used = new ArrayList<>();
            for (int i = 0; i < r; i++) {
                int[] e = new int[r];
                e[i] = 1;
                used.add(e);
            }

            int col = 0;
            for (int num = 1; num < (1 << r) && col < k; num++) {
                int[] vec = new int[r];
                for (int b = 0; b < r; b++) {
                    vec[b] = (num >> b) & 1;
                }

                // Проверяем, не является ли этот вектор уже использованным (единичным)
                boolean isUsed = false;
                for (int[] u : used) {
                    if (Arrays.equals(u, vec)) {
                        isUsed = true;
                        break;
                    }
                }
                if (!isUsed) {
                    for (int i = 0; i < r; i++) {
                        H[i][col] = vec[i];
                    }
                    used.add(vec);
                }
                col++;
            }
        }
    }
}

```

```

        col++;
    }
}
}

/**
 * Кодирование: systematic form
 * Информационные биты идут первыми, проверочные — в конце.
 * Проверочные биты вычисляются так, чтобы  $H * c^T = 0$ 
 */
String encode(String infoBits) {
    if (infoBits.length() != k || !infoBits.matches("[01]+")) {
        throw new IllegalArgumentException("Нужно ровно " + k + " бит (0/1)");
    }

    int[] c = new int[n];
    // Копируем информационные биты
    for (int i = 0; i < k; i++) {
        c[i] = infoBits.charAt(i) - '0';
    }

    // Вычисляем проверочные биты (последние r позиций)
    // Из условия  $H * c^T = 0 \rightarrow$  для каждой строки H:
    //  $\text{sum}(H[i][j] * c[j]) = 0 \pmod{2}$ 
    //  $\rightarrow c[k+i] = \text{sum}(H[i][j] * c[j])$  для  $j = 0..k-1$ 
    for (int i = 0; i < r; i++) {
        int sum = 0;
        for (int j = 0; j < k; j++) {
            sum ^= H[i][j] * c[j];
        }
        c[k + i] = sum;
    }

    StringBuilder sb = new StringBuilder();
    for (int bit : c) sb.append(bit);
    return sb.toString();
}

/**
 * Вычисление синдрома  $S = H * y^T$ 
 */
int[] calculateSyndrome(String received) {
    int[] y = new int[n];
    for (int i = 0; i < n; i++) {
        y[i] = received.charAt(i) - '0';
    }

    int[] syndrome = new int[r];
    for (int i = 0; i < r; i++) {
        int sum = 0;
        for (int j = 0; j < n; j++) {
            sum ^= H[i][j] * y[j];
        }
        syndrome[i] = sum;
    }
    return syndrome;
}

/**
 * Декодирование по синдрому
 */
static class DecodeResult {
    boolean hasError;
    boolean corrected;
    int errorPosition = -1; // позиция ошибки (0-based), -1 если нет
    String syndromeStr;
    String correctedWord;
    String message;
}

```

```

DecodeResult decode(String received) {
    DecodeResult res = new DecodeResult();
    if (received.length() != n) {
        res.message = "Неверная длина принятого слова";
        return res;
    }

    int[] syndrome = calculateSyndrome(received);
    res.syndromeStr = Arrays.toString(syndrome).replaceAll("[\\[\\], ]", "");

    // Если синдром нулевой — ошибок нет
    boolean allZero = true;
    for (int s : syndrome) {
        if (s != 0) {
            allZero = false;
            break;
        }
    }

    if (allZero) {
        res.hasError = false;
        res.corrected = true;
        res.correctedWord = received;
        res.message = "Синдром нулевой. Ошибок не обнаружено.";
        return res;
    }

    res.hasError = true;

    // Ищем столбец матрицы H, который совпадает с синдромом
    // Номер этого столбца = позиция ошибки
    for (int col = 0; col < n; col++) {
        boolean match = true;
        for (int i = 0; i < r; i++) {
            if (H[i][col] != syndrome[i]) {
                match = false;
                break;
            }
        }
        if (match) {
            res.errorPosition = col;
            break;
        }
    }

    if (res.errorPosition != -1) {
        // Исправляем ошибку
        char[] bits = received.toCharArray();
        bits[res.errorPosition] = (bits[res.errorPosition] == '0') ? '1' : '0';
        res.correctedWord = new String(bits);
        res.corrected = true;
        res.message = String.format(
            "Обнаружена одиночная ошибка в позиции %d (нумерация с 0). Исправлено.",
            res.errorPosition);
    } else {
        // Синдром не совпал ни с одним столбцом → двойная (или большей кратности) ошибка
        res.corrected = false;
        res.correctedWord = received;
        res.message = "Синдром ненулевой, но не соответствует ни одному столбцу H. " +
            "Вероятно, произошла ошибка кратности ≥ 2. Исправление невозможно.";
    }

    return res;
}

// Красивый вывод матрицы H
void printH() {
    System.out.println("\nПроверочная матрица H (" + r + " × " + n + "):");
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < n; j++) {

```

```

        System.out.print(H[i][j] + " ");
        if (j == k - 1) System.out.print("| "); // граница информационных и проверочных
    }
    System.out.println();
}
System.out.println("левая часть — P, правая — единичная матрица I_r");
}

void printParameters() {
    System.out.printf("Параметры кода: (n, k, r) = (%d, %d, %d), d_min = 3%n", n, k, r);
    System.out.printf("Скорость кода R = k/n = %.3f%n", (double) k / n);
    System.out.printf("Может исправлять t = %d ошибку, обнаруживать до %d ошибок%n",
        1, 2);
}

// ===== Вспомогательные методы =====
static String textToBinary(String text) {
    StringBuilder sb = new StringBuilder();
    for (char c : text.toCharArray()) {
        String bin = Integer.toBinaryString(c);
        // Дополняем до 8 бит (ASCII)
        while (bin.length() < 8) bin = "0" + bin;
        sb.append(bin);
    }
    return sb.toString();
}

static String randomBinary(int len) {
    Random rnd = new Random();
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < len; i++) sb.append(rnd.nextInt(2));
    return sb.toString();
}

static String introduceErrors(String codeword, int multiplicity) {
    char[] bits = codeword.toCharArray();
    Random rnd = new Random();
    Set<Integer> positions = new HashSet<>();
    while (positions.size() < multiplicity && positions.size() < bits.length) {
        positions.add(rnd.nextInt(bits.length));
    }
    System.out.print("Инвертированы позиции: ");
    for (int pos : positions) {
        bits[pos] = (bits[pos] == '0') ? '1' : '0';
        System.out.print(pos + " ");
    }
    System.out.println();
    return new String(bits);
}

// ===== Главное меню =====
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    HammingCode code = null;
    String currentInfo = null;
    String currentCodeword = null;

    System.out.println("==== Лабораторная работа №4. Код Хемминга ====");
    System.out.println("Часть 1 лабораторной работы №2\n");

    while (true) {
        System.out.println("-----");
        System.out.println("1. Ввести информационное сообщение (текст или биты)");
        System.out.println("2. Показать параметры кода и матрицу H");
        System.out.println("3. Закодировать");
        System.out.println("4. Внести ошибки (0, 1 или 2)");
        System.out.println("5. Декодировать (вычислить синдром и исправить)");
        System.out.println("6. Полный пример (текст → код → ошибка → исправление)");
        System.out.println("0. Выход");
    }
}

```

```

System.out.print("Выбор: ");

int choice;
try {
    choice = Integer.parseInt(sc.nextLine().trim());
} catch (Exception e) {
    System.out.println("Введите число!");
    continue;
}

switch (choice) {
    case 1 -> {
        System.out.println("Выберите способ ввода:");
        System.out.println(" а) Текст (будет переведён в ASCII-биты)");
        System.out.println(" б) Двоичная строка");
        System.out.println(" в) Случайная двоичная строка заданной длины");
        System.out.print("Ваш выбор (a/b/c): ");
        String mode = sc.nextLine().trim().toLowerCase();

        String bits;
        if (mode.equals("a")) {
            System.out.print("Введите текст (не меньше 2-3 символов): ");
            String text = sc.nextLine();
            bits = textToBinary(text);
            System.out.println("Двоичное представление (" + bits.length() + " бит):");
            System.out.println(bits);
        } else if (mode.equals("b")) {
            System.out.print("Введите двоичную строку: ");
            bits = sc.nextLine().trim();
        } else {
            System.out.print("Длина в битах ( $\geq 16$ ): ");
            int len = Integer.parseInt(sc.nextLine().trim());
            bits = randomBinary(len);
            System.out.println("Сгенерировано: " + bits);
        }

        if (bits.length() < 16) {
            System.out.println("Внимание: по заданию желательно  $\geq 16$  бит. Продолжаем...");
        }
        if (!bits.matches("[01]+")) {
            System.out.println("Ошибка: только символы 0 и 1!");
            break;
        }

        currentInfo = bits;
        code = new HammingCode(bits.length());
        System.out.println("Информационное слово принято. Параметры кода пересчитаны.");
        code.printParameters();
    }
    case 2 -> {
        if (code == null) {
            System.out.println("Сначала введите информационное слово (пункт 1)");
            break;
        }
        code.printParameters();
        code.printH();
    }
    case 3 -> {
        if (code == null || currentInfo == null) {
            System.out.println("Сначала введите информационное слово");
            break;
        }
        try {
            currentCodeword = code.encode(currentInfo);
            System.out.println("Информационное слово (" + code.k + " бит):");
            System.out.println(currentInfo);
            System.out.println("\nКодовое слово Xn (" + code.n + " бит):");
            System.out.println(currentCodeword);
            System.out.println("(первые " + code.k + " бит — информационные, последние " + code.r + " — проверочные)");
        }
    }
}

```

```

    } catch (Exception e) {
        System.out.println("Ошибка кодирования: " + e.getMessage());
    }
}
case 4 -> {
    if (currentCodeword == null) {
        System.out.println("Сначала закодируйте слово (пункт 3)");
        break;
    }
    System.out.print("Кратность ошибки (0, 1 или 2): ");
    try {
        int mult = Integer.parseInt(sc.nextLine().trim());
        if (mult < 0 || mult > 2) {
            System.out.println("Рекомендуется 0–2. Продолжаем...");
        }
        String errored = introduceErrors(currentCodeword, mult);
        System.out.println("Исходное : " + currentCodeword);
        System.out.println("С ошибками: " + errored);
        currentCodeword = errored;
    } catch (Exception e) {
        System.out.println("Некорректный ввод");
    }
}
case 5 -> {
    if (code == null || currentCodeword == null) {
        System.out.println("Нет данных для декодирования");
        break;
    }
    HammingCode.DecodeResult res = code.decode(currentCodeword);
    System.out.println("\n--- Результат декодирования ---");
    System.out.println("Синдром S = " + res.syndromeStr);
    System.out.println(res.message);
    if (res.corrected && res.hasError) {
        System.out.println("Исправленное слово: " + res.correctedWord);
    }
}
case 6 -> {
    // Полный демонстрационный прогон
    System.out.print("Введите короткий текст для демонстрации: ");
    String text = sc.nextLine();
    String bits = textToBinary(text);
    System.out.println("\n1. Информационные биты (" + bits.length() + " шт.): " + bits);

    code = new HammingCode(bits.length());
    code.printParameters();
    code.printH();

    String cw = code.encode(bits);
    System.out.println("\n2. Закодированное слово: " + cw);

    System.out.println("\n3. Вносим одну случайную ошибку:");
    String errored = introduceErrors(cw, 1);
    System.out.println(" Принято: " + errored);

    System.out.println("\n4. Декодирование:");
    HammingCode.DecodeResult res = code.decode(errored);
    System.out.println(" Синдром: " + res.syndromeStr);
    System.out.println(" " + res.message);
    if (res.corrected) {
        System.out.println(" После исправления: " + res.correctedWord);
        // Проверяем, совпало ли с исходным
        if (res.correctedWord.equals(cw)) {
            System.out.println(" ✓ Успешно восстановлено исходное кодовое слово!");
        }
    }
}
case 0 -> {
    System.out.println("Выход.");
    return;
}
}

```

```

        default -> System.out.println("Неверный пункт");
    }
    System.out.println();
}
}
}

```

## Часть 2:

Разработать собственное приложение, которое позволяет выполнять следующие операции: 1) вписывать произвольное двоичное представление информационного слова  $X_k$  (кодируемой информации) длиной  $K$  битов в двумерную матрицу размерностью в соответствии с вариантом либо в трехмерную матрицу в соответствии с вариантом (указаны в табл. 5.2); 2) вычислять проверочные биты (биты паритетов): а) по двум; б) по трем; в) по четырем направлениям (группам паритетов); 3) формировать кодовое слово  $X$ , присоединением избыточных символов к информационному слову; 4) генерировать ошибку произвольной кратности ( $i, i > 0$ ), распределенную случайным (фазом среди символов слова  $X$ , в результате чего формируется кодовое слово  $U$ ; 5) определять местоположение ошибочных символов итеративным кодом в слове  $U$  в соответствии с используемыми группами паритетов по пункту (2) и исправлять ошибочные символы (результат исправления слово  $U$ ); б) выполнять анализ корректирующей способности используемого кода (количественная оценка) путем сравнения соответствующих слов  $X$  и  $U$ ; результат анализа может быть представлен в виде отношения общего числа сгенерированных кодовых слов с ошибками определенной одинаковой кратности (с одной ошибкой, с двумя ошибками и т. д.) к числу кодовых слов, содержащих ошибки этой кратности, которые правильно обнаружены и которые правильно скорректированы.

| Вариант | Длина информационного слова (бит), $k$ | $k_1$ | $k_2$ | $z$ | Количество групп паритетов |
|---------|----------------------------------------|-------|-------|-----|----------------------------|
| 3       | 24                                     | 4     | 6     | –   | 2; 3                       |
|         |                                        | 3     | 8     | –   | 2; 3                       |
|         |                                        | 3     | 3     | 4   | 2; 3; 4; 5                 |
|         |                                        | 6     | 2     | 2   | 2; 3; 4; 5                 |

```

import java.util.*;

/**
 * Лабораторная работа №5. Итеративные коды.
 * Вариант 3:  $k=24, k_1=4, k_2=6, 3$  группы паритетов
 * (строки + столбцы + общий паритет).
 */
public class Lab2 {

    // ===== Параметры варианта =====
    static final int K1 = 4;    // строка

```

```

static final int K2 = 6;      // столбцов
static final int K = K1 * K2; // 24 информационных бита
static final int N = K + K1 + K2 + 1; // 24 + 4 + 6 + 1 = 35

// ===== Основной класс кода =====
static class IterativeCode {
    int[][] info;      // информационная матрица [K1][K2]
    int[] rowParity;    // паритеты строк [K1]
    int[] colParity;    // паритеты столбцов [K2]
    int overallParity;  // общий паритет (паритет паритетов)

    // ----- Кодирование -----
    void encode(String binary) {
        if (binary.length() != K || !binary.matches("[01]+")) {
            throw new IllegalArgumentException("Нужно ровно " + K + " бит (0/1)");
        }

        info = new int[K1][K2];
        int idx = 0;
        for (int i = 0; i < K1; i++) {
            for (int j = 0; j < K2; j++) {
                info[i][j] = binary.charAt(idx++) - '0';
            }
        }

        // Паритеты строк (чётность)
        rowParity = new int[K1];
        for (int i = 0; i < K1; i++) {
            int sum = 0;
            for (int j = 0; j < K2; j++) sum ^= info[i][j];
            rowParity[i] = sum;
        }

        // Паритеты столбцов
        colParity = new int[K2];
        for (int j = 0; j < K2; j++) {
            int sum = 0;
            for (int i = 0; i < K1; i++) sum ^= info[i][j];
            colParity[j] = sum;
        }

        // Общий паритет (паритет всех строковых паритетов + всех столбцовых)
        // Можно считать и как паритет всей информационной матрицы + паритетов
        int sum = 0;
        for (int p : rowParity) sum ^= p;
        for (int p : colParity) sum ^= p;
        overallParity = sum;
    }

    // Получить полное кодовое слово (информация + паритеты)
    String getCodeword() {
        StringBuilder sb = new StringBuilder();
        // Информационные биты
        for (int i = 0; i < K1; i++)
            for (int j = 0; j < K2; j++)
                sb.append(info[i][j]);
        // Строковые паритеты
        for (int p : rowParity) sb.append(p);
        // Столбцовые паритеты
        for (int p : colParity) sb.append(p);
        // Общий
        sb.append(overallParity);
        return sb.toString();
    }
}

```

```

}

// ----- Декодирование и исправление -----
static class DecodeResult {
    boolean corrected;
    int errorRow = -1, errorCol = -1;
    String message;
    String correctedCodeword;
}

DecodeResult decode(String received) {
    DecodeResult res = new DecodeResult();
    if (received.length() != N) {
        res.message = "Неверная длина принятого слова";
        return res;
    }

    // Разбираем принятое слово
    int[][] rInfo = new int[K1][K2];
    int[] rRowP = new int[K1];
    int[] rColP = new int[K2];
    int rOverall;

    int idx = 0;
    for (int i = 0; i < K1; i++)
        for (int j = 0; j < K2; j++)
            rInfo[i][j] = received.charAt(idx++) - '0';

    for (int i = 0; i < K1; i++) rRowP[i] = received.charAt(idx++) - '0';
    for (int j = 0; j < K2; j++) rColP[j] = received.charAt(idx++) - '0';
    rOverall = received.charAt(idx) - '0';

    // Пересчитываем паритеты по принятой информации
    int[] calcRowP = new int[K1];
    for (int i = 0; i < K1; i++) {
        int sum = 0;
        for (int j = 0; j < K2; j++) sum ^= rInfo[i][j];
        calcRowP[i] = sum;
    }

    int[] calcColP = new int[K2];
    for (int j = 0; j < K2; j++) {
        int sum = 0;
        for (int i = 0; i < K1; i++) sum ^= rInfo[i][j];
        calcColP[j] = sum;
    }

    // Синдромы (несовпадения паритетов)
    int[] rowSyndrome = new int[K1];
    int[] colSyndrome = new int[K2];
    for (int i = 0; i < K1; i++) rowSyndrome[i] = calcRowP[i] ^ rRowP[i];
    for (int j = 0; j < K2; j++) colSyndrome[j] = calcColP[j] ^ rColP[j];

    // Ищем строки и столбцы с ошибкой
    List<Integer> badRows = new ArrayList<>();
    List<Integer> badCols = new ArrayList<>();
    for (int i = 0; i < K1; i++) if (rowSyndrome[i] == 1) badRows.add(i);
    for (int j = 0; j < K2; j++) if (colSyndrome[j] == 1) badCols.add(j);

    // Проверка общего паритета
    int calcOverall = 0;
    for (int p : calcRowP) calcOverall ^= p;
    for (int p : calcColP) calcOverall ^= p;

```

```

boolean overallFail = (calcOverall != rOverall);

// Логика исправления
if (badRows.isEmpty() && badCols.isEmpty()) {
    if (!overallFail) {
        res.message = "Ошибок не обнаружено. Слово принято корректно.";
        res.corrected = true;
    } else {
        res.message = "Обнаружена ошибка в общем паритете (или тройная ошибка).";
        res.corrected = false;
    }
} else if (badRows.size() == 1 && badCols.size() == 1) {
    // Классическая одиночная ошибка в информационной части
    int er = badRows.get(0);
    int ec = badCols.get(0);
    rInfo[er][ec] ^= 1; // исправляем
    res.errorRow = er;
    res.errorCol = ec;
    res.corrected = true;
    res.message = String.format("Обнаружена и исправлена одиночная ошибка в позиции
[%d][%d]", er, ec);
} else if (badRows.size() == 1 && badCols.isEmpty()) {
    // Ошибка в строковом паритете
    res.message = "Ошибка в строковом паритете строки " + badRows.get(0);
    res.corrected = true; // паритет можно просто пересчитать
} else if (badRows.isEmpty() && badCols.size() == 1) {
    res.message = "Ошибка в столбцовом паритете столбца " + badCols.get(0);
    res.corrected = true;
} else {
    res.message = String.format(
        "Обнаружена ошибка кратности > 1 (плохие строки: %s, столбцы: %s). Исправление
невозможно.",
        badRows, badCols);
    res.corrected = false;
}

// Собираем исправленное слово
StringBuilder sb = new StringBuilder();
for (int i = 0; i < KI; i++)
    for (int j = 0; j < K2; j++)
        sb.append(rInfo[i][j]);
// Паритеты оставляем как есть или пересчитываем — для простоты оставляем принятые
for (int p : rRowP) sb.append(p);
for (int p : rColP) sb.append(p);
sb.append(rOverall);
res.correctedCodeword = sb.toString();

return res;
}

// Красивый вывод матрицы
void printMatrix() {
    System.out.println("\nИнформационная матрица + паритеты:");
    System.out.print(" ");
    for (int j = 0; j < K2; j++) System.out.printf(" c%d ", j);
    System.out.println(" | Prow");
    System.out.println("  + " + "----".repeat(K2) + "+-----");

    for (int i = 0; i < KI; i++) {
        System.out.printf(" r%d |", i);
        for (int j = 0; j < K2; j++) System.out.printf(" %d ", info[i][j]);
        System.out.printf(" | %d%n", rowParity[i]);
    }
}

```

```

        System.out.println("  "+"-----".repeat(K2) + "+-----");
        System.out.print("Pcol");
        for (int j = 0; j < K2; j++) System.out.printf(" %d ", colParity[j]);
        System.out.printf("| %d (overall)%n", overallParity);
    }
}

// ===== Вспомогательные методы =====
static String randomBinary(int len) {
    Random rnd = new Random();
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < len; i++) sb.append(rnd.nextInt(2));
    return sb.toString();
}

static String introduceErrors(String codeword, int multiplicity) {
    char[] bits = codeword.toCharArray();
    Random rnd = new Random();
    Set<Integer> positions = new HashSet<>();
    while (positions.size() < multiplicity) {
        positions.add(rnd.nextInt(bits.length));
    }
    for (int pos : positions) {
        bits[pos] = (bits[pos] == '0') ? '1' : '0';
    }
    return new String(bits);
}

// ===== Главное меню =====
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    IterativeCode code = new IterativeCode();
    String currentInfo = null;
    String currentCodeword = null;

    System.out.println("=== Лабораторная работа №5. Итеративные коды ===");
    System.out.println("Вариант 3: k=24, матрица 4×6, 3 группы паритетов");
    System.out.println("Длина кодового слова n = " + N + " бит\n");

    while (true) {
        System.out.println("-----");
        System.out.println("1. Ввести / сгенерировать информационное слово");
        System.out.println("2. Закодировать и показать матрицу");
        System.out.println("3. Внести ошибки");
        System.out.println("4. Декодировать и исправить");
        System.out.println("5. Анализ корректирующей способности");
        System.out.println("0. Выход");
        System.out.print("Выбор: ");

        int choice;
        try {
            choice = Integer.parseInt(sc.nextLine().trim());
        } catch (Exception e) {
            System.out.println("Введите число!");
            continue;
        }

        switch (choice) {
            case 1 -> {
                System.out.print("Введите 24 бита (или Enter для случайной генерации): ");
                String input = sc.nextLine().trim();
                if (input.isEmpty()) {
                    currentInfo = randomBinary(K);
                }
            }
        }
    }
}

```

```

        System.out.println("Сгенерировано: " + currentInfo);
    } else {
        currentInfo = input;
    }
    try {
        code.encode(currentInfo);
        currentCodeword = code.getCodeword();
        System.out.println("Информационное слово принято.");
    } catch (Exception e) {
        System.out.println("Ошибка: " + e.getMessage());
        currentInfo = null;
    }
}
case 2 -> {
    if (currentInfo == null) {
        System.out.println("Сначала введите информационное слово (пункт 1)");
        break;
    }
    code.printMatrix();
    System.out.println("\nКодовое слово Xn (" + N + " бит):");
    System.out.println(currentCodeword);
}
case 3 -> {
    if (currentCodeword == null) {
        System.out.println("Сначала закодируйте слово");
        break;
    }
    System.out.print("Кратность ошибки (1, 2, 3...): ");
    try {
        int mult = Integer.parseInt(sc.nextLine().trim());
        String errored = introduceErrors(currentCodeword, mult);
        System.out.println("Исходное : " + currentCodeword);
        System.out.println("С ошибками: " + errored);
        currentCodeword = errored; // теперь работаем с испорченным
    } catch (Exception e) {
        System.out.println("Некорректный ввод");
    }
}
case 4 -> {
    if (currentCodeword == null) {
        System.out.println("Нет кодового слова");
        break;
    }
    IterativeCode.DecodeResult res = code.decode(currentCodeword);
    System.out.println("\nРезультат декодирования:");
    System.out.println(res.message);
    if (res.corrected) {
        System.out.println("Исправленное слово: " + res.correctedCodeword);
    }
}
case 5 -> {
    System.out.println("\n--- Анализ корректирующей способности ---");
    System.out.print("Количество испытаний: ");
    int trials = Integer.parseInt(sc.nextLine().trim());
    System.out.print("Кратность ошибки: ");
    int mult = Integer.parseInt(sc.nextLine().trim());

    int detected = 0, corrected = 0;
    for (int t = 0; t < trials; t++) {
        String info = randomBinary(K);
        code.encode(info);
        String cw = code.getCodeword();
        String errored = introduceErrors(cw, mult);

```

```

        IterativeCode.DecodeResult res = code.decode(errorred);
        if (!res.message.contains("не обнаружено")) detected++;
        if (res.corrected && res.message.contains("исправлена")) corrected++;
    }
    System.out.printf("Испытаний: %d, кратность ошибки: %d%n", trials, mult);
    System.out.printf("Обнаружено: %d (%.1f%%)%n", detected, 100.0 * detected / trials);
    System.out.printf("Исправлено: %d (%.1f%%)%n", corrected, 100.0 * corrected / trials);
    System.out.println("Для одиночных ошибок код должен исправлять ~100%");
}
case 0 -> {
    System.out.println("Выход.");
    return;
}
default -> System.out.println("Неверный пункт");
}
System.out.println();
}
}
}

```

Вывод: в ходе лабораторной работы освоены принципы избыточного кодирования на примере кода Хемминга и итеративных кодов. Реализованы алгоритмы кодирования, декодирования и исправления одиночных ошибок. Практически подтверждена корректирующая способность обоих методов и проведён сравнительный анализ их параметров и эффективности.